# 04_Module_4_Monitoring_Observability

# Module 4 — Monitoring & Observability

## Duration: 8 Hours

---

## 4.1 Monitoring vs Observability

| Monitoring | Observability |
|---|---|
| Tells you something is wrong | Helps you understand why |
| Based on known failure modes | Works for unknown failures |
| Requires you to predict thresholds | Lets you explore unexpected behaviour |
| “The alert fired” | “I can query the system to find the cause” |

**Analogy:** - Monitoring is the car dashboard: the check engine light tells you something is wrong - Observability is the diagnostic computer: it tells you exactly which sensor, which cylinder, and why

**You need both.** The alert wakes you up (monitoring). The observability tools help you fix it fast.

---

## 4.2 The Three Pillars of Observability

### Pillar 1: Metrics

Numerical measurements collected over time. Best for tracking trends, triggering alerts, and powering dashboards.

**Prometheus metric format:**

```
atm_withdrawal_success_total{location="Lagos_Island", atm_id="ATM-001"} 9950
atm_withdrawal_total{location="Lagos_Island", atm_id="ATM-001"} 10000
http_request_duration_seconds_bucket{service="mobile-api", le="0.5"} 4500
http_request_duration_seconds_bucket{service="mobile-api", le="1.0"} 8200
http_request_duration_seconds_bucket{service="mobile-api", le="+Inf"} 10000
```

### Pillar 2: Logs

Timestamped text records of events inside the system. Best for debugging specific transactions, audit trails, and incident investigation.

**Structured logging example:**

```
{
  "timestamp": "2026-02-23T14:32:01.123Z",
  "level": "ERROR",
  "service": "atm-service",
  "atm_id": "ATM-001",
  "transaction_id": "TXN-20260223-9921",
```

```
  "error_code": "CR_047",
  "message": "Card reader failure after 3 retries",
  "stack_trace": "at com.union.atm.CardReader.readCard():142"
}
```

**At scale:** don't read logs manually. Use ELK, Loki, or similar to search and filter.

### Pillar 3: Traces

Records of a request's complete journey through multiple services. Best for identifying bottlenecks in distributed systems.

**Trace example (Jaeger/OpenTelemetry):**

```
Service: Transfer API       → 120ms
  ├── Auth Service          → 45ms  ✓
  ├── Core Banking API      → 3.2s  ✗ BOTTLENECK
  ├── Fraud Detection       → 80ms  ✓
  └── SMS Notification      → 150ms ✓
Total: 3.6s
```

**When to trace:** any request that crosses service boundaries. If a transfer touches 5 services, each one could be the bottleneck — only tracing tells you which.

---

## 4.3 The RED and USE Methods

Two complementary frameworks for choosing what to measure:

### RED Method (for services/request-path)

| Metric | What It Measures | Questions It Answers |
|---|---|---|
| **Rate** | Requests per second | Is traffic changing? |
| **Errors** | Failed requests per second | Is the service working? |
| **Duration** | Request latency distribution | Is it fast enough? |

**Prometheus implementation:**

```
# Rate
rate(http_requests_total[5m])

# Errors
rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m])

# Duration (p99)
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))
```

### USE Method (for infrastructure/resources)

| Metric | What It Measures | Questions It Answers |
|---|---|---|
| **Utilisation** | % of resource being used | Is the resource maxed out? |
| **Saturation** | Queue length or backlog | Is there more work than capacity? |
| **Errors** | Count of error events | Is the device failing? |

**Banking USE examples:**

| Resource | Utilisation | Saturation | Errors |
|---|---|---|---|
| CPU | `node_cpu_seconds_total{mode="user"}` | Load average | Machine check exceptions |
| Memory | `node_memory_MemTotal_bytes − node_memory_MemAvailable_bytes` | OOM kill count | OOM events |
| Disk | `node_filesystem_size_bytes − node_filesystem_free_bytes` | I/O queue depth | Disk errors |
| Network | Interface bandwidth utilisation | Interface drops | Interface errors |

---

# 4.4 Advanced PromQL

## Prediction

Forecast when a resource will run out:

```
# Disk full in how many hours?
predict_linear(node_filesystem_free_bytes{mountpoint="/"}[6h], 3600 * 24 * 7)

# Will disk be full in 7 days?
predict_linear(node_filesystem_free_bytes{mountpoint="/"}[6h], 3600 * 24 * 7) < 0
```

## Rate vs Irate

```
# rate: average over window (smoother, use for alerting)
rate(http_requests_total[5m])

# irate: instantaneous rate (spikier, use for dashboards)
irate(http_requests_total[5m])
```

## SLO Burn Rate Calculation

```
# Current burn rate (ratio of actual to allowed error rate)
(
  1 - (
    sum(rate(http_requests_total{status!~"2.."}[1h]))
    /
    sum(rate(http_requests_total[1h]))
  )
) / (1 - 0.999)  # SLO = 99.9%
```

---

# 4.5 Instrumentation with OpenTelemetry

OpenTelemetry (OTel) is the industry standard for generating metrics, logs, and traces.

**Python instrumentation example:**

```
from opentelemetry import trace
from opentelemetry.exporter.jaeger import JaegerExporter
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
```

```python
# Set up tracing
provider = TracerProvider()
jaeger_exporter = JaegerExporter(
    agent_host_name="jaeger",
    agent_port=6831,
)
provider.add_span_processor(BatchSpanProcessor(jaeger_exporter))
trace.set_tracer_provider(provider)
tracer = trace.get_tracer(__name__)

# Instrument a transfer
with tracer.start_as_current_span("process_transfer") as span:
    span.set_attribute("transfer_id", "TXN-12345")
    span.set_attribute("amount", 50000)
    span.set_attribute("currency", "NGN")

    with tracer.start_as_current_span("deduct_balance") as child:
        child.set_attribute("account", "****1234")
        # ... business logic ...

    with tracer.start_as_current_span("send_sms") as child:
        child.set_attribute("phone", "****7890")
        # ... SMS logic ...
```

---

# 4.6 Alerting Design

## The Problem

Most teams suffer from **alert fatigue** — too many alerts, too many false positives, too many that aren't actionable.

**Signs of alert fatigue:** - Engineers silence alerts without investigating - The same alert fires daily for months with no fix - On-call engineers are exhausted - Real incidents are missed because they look like noise

## Principles of Good Alerting

1. **Alert on symptoms, not causes**
   - ❌ CPU > 90% (cause)
   - ✅ Error rate > SLO budget burn rate (symptom)
2. **Every alert must be actionable**
   - Ask: "What will the on-call engineer DO when this fires?"
   - If the answer is "nothing right now" — it's a ticket, not an alert
3. **Every alert must have a runbook**
   - The runbook is checked into Git, linked from the alert
   - It includes step-by-step response, escalation path, and expected outcomes
4. **Use severity levels**
   - **Critical** — user-facing impact, wake someone up
   - **Warning** — degraded but functional, investigate within the hour
   - **Info** — notable event, review during business hours
5. **Alert on burn rate, not static thresholds**
   - See Module 2 for multi-window burn rate alerting rules

## Runbook Example

```
# Runbook: ATM_HIGH_ERROR_RATE

## Alert Condition
Error rate > 2x SLO burn rate for 5 minutes (1-hour window)

## Severity
Critical

## Steps

1. **ACKNOWLEDGE** — Acknowledge the alert in PagerDuty

2. **CHECK DASHBOARD** — Open Grafana dashboard "ATM Services"
   - Is the error rate elevated across ALL ATMs or a subset?
   - Is latency also elevated?

3. **IDENTIFY AFFECTED ATMS**
   - Query: `atm_errors_total{location=~".*"}`
   - Check if specific locations are affected

4. **CHECK LOGS** — Open Kibana
   - Filter: `service:"atm-service" AND level:"ERROR"`
   - Time range: last 15 minutes
   - Identify the error code pattern

5. **RESPOND BY ERROR CODE:**
   - `CR_047` (card reader failure) → Call ATM hardware team: 0802-XXX-XXXX
   - `NW_001` (network timeout) → Check network connectivity to affected ATMs
   - `DB_ERR` (database error) → Check core banking API status
   - Unknown pattern → Escalate to SRE lead

6. **ESCALATE IF:**
   - > 10 ATMs affected → Page SRE lead
   - > 50 ATMs affected → Open SEV-1 incident
   - Any ATM has been down > 30 min → Page SRE lead

7. **RESOLVE** — Once error rate is below threshold for 15 min
   - Confirm in Grafana
   - Close alert in PagerDuty
   - Log incident summary
```

---

# 4.7 On-Call Management

## Rotation Design

| Rule | Why |
|---|---|
| Max 1 week in 4 on-call | Prevents burnout |
| Max 2 pages per shift | More pages means something needs fixing |
| Follow-the-sun rotation | Distributes night shifts fairly |
| Secondary on-call | Backup if primary doesn't respond |

## Escalation Policy

```
Primary on-call ← alert fires
  └── No response in 10 min → Secondary on-call
```

```
        └── No response in 10 min → SRE Team Lead
              └── No response in 10 min → CTO/Head of Technology
```

---

## 4.8 Classroom Activity: Design a Monitoring Stack

**Time:** 1 hour

Design a complete monitoring stack for Union Bank's mobile banking app:

1. **Choose your tools** — Prometheus + Grafana? Datadog? ELK? Loki? Justify each choice
2. **Define 5 RED metrics** — Rate, Error, Duration for the mobile API
3. **Define 3 USE metrics** — for the infrastructure supporting it
4. **Write 3 burn rate alerts** — with windows, thresholds, and runbooks
5. **Design 1 dashboard** — what does a healthy service look like at a glance?
6. **Design the on-call rotation** — team of 8 engineers, 24/7 coverage

---

## Key Terms

| Term | Definition |
|---|---|
| **RED Method** | Rate, Errors, Duration — the three metrics for service monitoring |
| **USE Method** | Utilisation, Saturation, Errors — the three metrics for infrastructure |
| **OpenTelemetry** | Industry standard for generating metrics, logs, and traces |
| **PromQL** | Prometheus Query Language |
| **Runbook** | Step-by-step instructions for responding to an alert |
| **Alert Fatigue** | When too many alerts cause engineers to ignore or silence them |
| **p99 Latency** | The response time that 99% of requests complete within |
| **Symptom-based alerting** | Alerting based on user-facing impact, not internal causes |

---

## Further Reading

- Google SRE Book, Chapter 6 — Monitoring Distributed Systems
- The RED Method — Tom Wilkie
- The USE Method — Brendan Gregg
- OpenTelemetry Documentation
- PromQL Documentation